OPÉRATION KUBE-SPACE : MISSION DE SAUVETAGE

VOTRE MISSION

Statut : URGENT
Lieu : Station Spatiale K8S-Alpha
Problème : Panne totale des systèmes de contrôle
Votre rôle : Ingénieur(e) envoyé(e) en urgence

La station K8S-Alpha dérive dans l'espace. Les anciens systèmes sont tombés en panne. Vous devez **redémarrer les moteurs de contrôle** en utilisant la technologie Kubernetes pour orchestrer les systèmes critiques de la station.

Durée de la mission : 55 min
Équipement : Minikube, Kubectl et le courage ✊

⚠️ ***Prérequis :*** *Si vous n'avez pas encore installé les outils nécessaires (Minikube, Kubectl), rendez-vous à la section* ***"Aide Rapide"*** *en fin de TP pour trouver les commandes d'installation avant de commencer.*

Ce TP a été réalisé par ***Lysa Matmar*** *et* ***Omar Samy Ahmed Hussein*** *de* ***Polytech Montpellier***

PHASE 1 : REDÉMARRAGE D'URGENCE

PARTIE COURS

1.1 - Qu'est-ce que Kubernetes ?

Kubernetes (K8s) est un orchestrateur de conteneurs. Il automatise le déploiement, la mise à l'échelle et la gestion d'applications conteneurisées.

> Si Docker est une capsule spatiale individuelle, Kubernetes est le centre de contrôle qui gère toute une flotte de capsules.

1.2 - Le Pod : L'unité de base

Un **Pod** est la plus petite unité déployable dans Kubernetes. Il contient un ou plusieurs conteneurs Docker qui partagent :

- La même adresse IP
- Le même espace de stockage
- Le même réseau

Caractéristiques importantes

- **Éphémère :** Un Pod peut mourir à tout moment
- **IP dynamique :** À chaque redémarrage, le Pod change d'adresse IP
- **Pas de self-healing** : Si vous créez un Pod seul et qu'il meurt, il ne redémarre pas automatiquement

> **Pod = Capsule spatiale autonome**
>
> - Elle peut fonctionner seule
> - Si elle explose, elle ne se répare pas toute seule
> - Elle change de position (IP) quand elle redémarre

Structure YAML d'un Pod

```yaml
apiVersion: v1           # Version de l'API Kubernetes
kind: Pod                # Type de ressource
metadata:
  name: mon-pod          # Nom unique du Pod
  labels:                # Étiquettes pour l'identifier
    app: mon-app
spec:
  containers:            # Liste des conteneurs
  - name: mon-conteneur
    image: nginx         # Image Docker à utiliser
    ports:
    - containerPort: 80 # Port exposé par le conteneur
```

1.3 - Le Service : Le point d'accès stable

TOC TOC TOC, un petit problème

SI les Pods ont des IPs qui changent constamment. Comment accéder à une application de manière stable ?

Solution : Le Service

Un **Service** est une abstraction qui définit un point d'accès stable vers un ensemble de Pods.

Il existe 3 types de Services principaux :

Type	Usage	Accès
ClusterIP	Communication interne uniquement	Uniquement dans le cluster
NodePort	Exposer sur un port du nœud	Accès externe via IP:Port
LoadBalancer	Pour le cloud (AWS, GCP, Azure)	IP publique automatique

Pour Minikube, on utilise NodePort.

> **Service = Standardiste téléphonique**
> - Vous appelez toujours le même numéro (IP fixe du Service)
> - Le standardiste redirige vers le bon employé (Pod disponible)
> - Même si l'employé change de bureau (IP du Pod), le numéro reste le même

Structure YAML d'un Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mon-service
spec:
  type: NodePort                # Type d'exposition
  selector:                     # Comment trouver les Pods
    app: mon-app                # Recherche les Pods avec ce label
  ports:
  - protocol: TCP
    port: 80                    # Port du Service
    targetPort: 80              # Port du conteneur
    nodePort: 30080             # Port externe (30000-32767)
```

Le mécanisme du Selector

⚡**TRÈS IMPORTANT : Le Service utilise les labels pour trouver les Pods.**

```
Pod labels:app=mon-app
          ↓
Service selector:app=mon-app  ← Doit correspondre !
```

Si le selector ne correspond pas aux labels → Le Service ne trouve aucun Pod !

PARTIE TP

Objectif : Déployer une application web (jeu 2048) dans un Pod et l'exposer via un Service pour y accéder depuis votre navigateur, dans ce TP, on va utiliser l'image docker : marcincuber/2048-game:latest

Étape 1.1 - Vérifier l'environnement

un souci, allez a la section (ajouter a la section SOS)

```
# Démarrer Minikube (si pas déjà fait)
minikube start

# Vérifier que le cluster fonctionne
kubectl cluster-info

# Vérifier les nœuds disponibles
kubectl get nodes
```

✅ **Résultat attendu :** Vous devez voir un nœud en état « Ready ».

Étape 1.2 - Créer le fichier Pod

Créez un fichier `pod-moteur.yaml` À COMPLÉTER :

```
apiVersion: v1
kind: ____
metadata:
  name: moteur-controle
  labels:
    app: _____
    mission: sauvetage
spec:
  containers:
  - name: interface-controle
    image: _____
    ports:
    - containerPort: _____
```

Votre mission :

1. Complétez les champs marqués _____
2. Créez le Pod : `kubectl apply -f pod-moteur.yaml`
3. Vérifiez le statut avec les commandes ci-dessous

```
# Voir tous les Pods
kubectl get pods

# Détails complets du Pod
kubectl describe pod moteur-controle

# Suivre les logs ( pour le moment y'en a pas)
kubectl logs moteur-controle
```

Résultat attendu :

- STATUS = `Running`
- READY = `1/1`

Étape 1.3 - Créer le fichier Service

Le Pod tourne, mais il n'est **pas accessible de l'extérieur**. Créons un Service.

Créez un fichier `service-moteur.yaml` À COMPLÉTER :

```
apiVersion: v1
kind: Service
metadata:
  name: service-moteur
spec:
  type: _____
  selector:
    app: _____
  ports:
  - protocol: TCP
    port: 80
    targetPort: _____
    nodePort: 30080
```

Votre mission :

4. Complétez les champs
5. Créez le Service : `kubectl apply -f service-moteur.yaml`
6. Vérifiez avec les commandes ci-dessous

```
# Lister les Services
kubectl get services

# Détails du Service
kubectl describe service service-moteur

# Vérifier les endpoints (IPs des Pods ciblés)
kubectl get endpoints service-moteur
```

✅ **Résultat attendu** : La colonne ENDPOINTS ne doit PAS être vide.

Étape 1.4 - Accéder à l'application

```
# Obtenir l'URL complète
minikube service service-moteur --url
```

Copiez l'URL et ouvrez-la dans votre navigateur

> ✅ **Résultat attendu** : Vous voyez le jeu 2048 !

STOP Expérience : Que se passe-t-il si le Pod meurt ?

```
# Supprimer le Pod
kubectl delete pod moteur-controle

# Attendre 10 secondes puis vérifier
kubectl get pods
```

> **Question :** Le Pod redémarre-t-il automatiquement ?
> Rafraichissez votre navigateur, que se passe-t-il ?

Conclusion partie 1 : Un Pod seul n'est **pas résilient**. Il nous faut un contrôleur intelligent, allons vers la prochaine station : **Deployment (Phase 2)**.

PHASE 2 : SYSTÈMES REDONDANTS ET RÉSILIENCE

PARTIE COURS

2.1 - RAPPEL : Le problème des Pods isolés

Ce que nous avons appris :

- Un Pod seul fonctionne
- Si le Pod meurt → Il disparaît pour toujours
- Pas de réplication automatique
- Pas de self-healing

Besoin : Un mécanisme qui garantit qu'un nombre désiré de Pods tourne en permanence.

2.2 - Le Deployment : Le contrôleur intelligent

Un **Deployment** est un contrôleur qui gère un ensemble de Pods identiques (appelés replicas).

Responsabilités du Deployment

7. **Création :** Lance le nombre désiré de Pods
8. **Surveillance :** Vérifie en permanence l'état des Pods
9. **Self-Healing** : Si un Pod meurt → Il en recrée un automatiquement
10. **Scaling** : Augmente ou réduit le nombre de replicas
11. **Rolling Updates :** Met à jour les Pods progressivement sans interruption

Deployment = Chef d'équipe spatial

- Vous lui dites : « Je veux 3 astronautes opérationnels »
- Il compte en permanence : 1... 2... 3
- Si l'un disparaît (accident) → Il envoie immédiatement un remplaçant
- **Il garantit toujours le nombre exact demandé**

2.3 - Structure YAML d'un Deployment

```
apiVersion: apps/v1          # API pour les contrôleurs
kind: Deployment
metadata:
  name: mon-deployment
spec:
  replicas: 3                # Nombre de Pods souhaités
  selector:
    matchLabels:             # Comment trouver ses Pods
      app: mon-app
  template:                  # Modèle de Pod à créer
    metadata:
      labels:
        app: mon-app         # Labels des Pods créés
    spec:
      containers:            # Identique à la spec d'un Pod
      - name: mon-conteneur
        image: marcincuber/2048-game:latest
        ports:
        - containerPort: 80
```

Points importants

1. Le champ `replicas`
   - Définit combien de copies du Pod vous voulez
   - Kubernetes maintient **toujours** ce nombre

2. Le champ `selector.matchLabels`
   - Permet au Deployment de reconnaître « ses » Pods
   - Doit correspondre aux labels dans `template.metadata.labels`

3. Le champ `template`
   - C'est le **modèle de Pod** que le Deployment utilisera
   - Identique à la définition d'un Pod classique

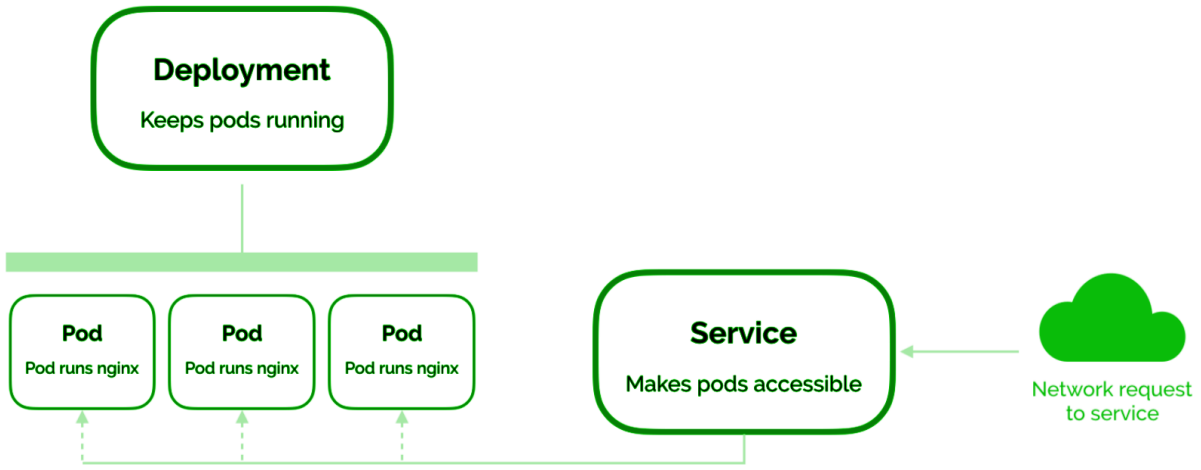
2.4 - Le Self-Healing en action

```
État initial : 3 Pods actifs
┌─────┐  ┌─────┐  ┌─────┐
│Pod1 │  │Pod2 │  │Pod3 │
└─────┘  └─────┘  └─────┘

Incident : Pod2 crash
┌─────┐  ┌─────┐  ┌─────┐
│Pod1 │  │ 💥  │  │Pod3 │
└─────┘  └─────┘  └─────┘

Kubernetes détecte : "Il manque un Pod !"

Auto-réparation : Création immédiate d'un nouveau Pod
┌─────┐  ┌─────┐  ┌─────┐
│Pod1 │  │Pod4 │  │Pod3 │
└─────┘  └─────┘  └─────┘
```

⏱ **Temps de réaction : Quelques secondes seulement !**

2.5 - Deployment + Service

Le Service continue de fonctionner avec un Deployment :



Avantage : Le Service répartit automatiquement le trafic entre les 3 Pods (load balancing).

PARTIE TP

Objectif : Transformer le Pod isolé en Deployment avec 3 replicas, puis tester la résilience en supprimant volontairement un Pod.

Étape 2.1 - Nettoyer le Pod isolé

Supprimons l'ancien Pod pour repartir proprement :

```
kubectl delete -f pod-moteur.yaml
```

Note : Le Service reste actif, mais il ne trouve plus de Pod (endpoints vides).

Étape 2.2 - Créer le fichier Deployment

Créez un fichier `deployment-moteur.yaml` À COMPLÉTER :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: moteur-controle-fleet
spec:
  replicas: _____
  selector:
    matchLabels:
      app: _____
  template:
    metadata:
      labels:
        app: _____
    spec:
      containers:
      - name: interface-controle
        image: marcincuber/2048-game:latest
        ports:
        - containerPort: 80
```

Votre mission :

12. Complétez les champs
13. Créez le Deployment : `kubectl apply -f deployment-moteur.yaml`
14. Vérifiez avec les commandes ci-dessous

```
# Voir le Deployment
kubectl get deployment

# Voir les Pods créés
kubectl get pods

# Plus de détails
kubectl describe deployment moteur-controle-fleet
```

✅ **Résultat attendu :**

- 3 Pods avec **des noms différents** (ex: `moteur-controle-fleet-abc123`)
- Tous en état `Running`

```
# Avant de continuer, vérifiez vos labels
kubectl get pods --show-labels
```

Notez bien la valeur du label "app=???"
Vous en aurez besoin à l'étape 2.4 !

Étape 2.3 - Vérifier que le Service fonctionne

```
# Vérifier les endpoints du Service
kubectl get endpoints service-moteur
```

✅ **Résultat attendu :** Vous devez voir 3 adresses IP (une par Pod).

```
# Obtenir l'URL
minikube service service-moteur --url
```

Ouvrez l'URL dans votre navigateur → **Le jeu 2048 est de retour !**

Étape 2.4 - ALERTE CHAOS : Détruire un Pod

C'est le moment le plus spectaculaire DE VOTRE MISSION !

Mode surveillance : Ouvrez un deuxième terminal et lancez :

```
# Remplacez "VOTRE-LABEL" par la valeur trouvée à l'étape 2.2
kubectl get pods -l app=VOTRE-LABEL -w
```

Le flag `-w` (watch) met à jour en temps réel.

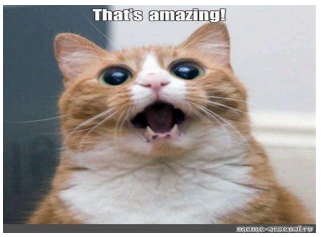
Dans le premier terminal :

```
# Listez les Pods
kubectl get pods -l app=moteur

# Copiez le nom d'un Pod (ex: moteur-controle-fleet-abc123)
# SUPPRIMEZ-LE
kubectl delete pod <NOM-DU-POD>
```

Observez le deuxième terminal :

- Le Pod supprimé passe en `Terminating`

- **IMMÉDIATEMENT**, un nouveau Pod apparaît en `ContainerCreating`
- Puis il passe en `Running`



C'est le self-healing (WOW AMAZING !)

Étape 2.5 - Vérifier la résilience du Service

- Rafraîchissez votre navigateur plusieurs fois
- Le jeu 2048 reste accessible !

Pourquoi ? Le Service redirige vers les 2 autres Pods encore actifs.

Expérience :

Augmentons le nombre de replicas :

```
kubectl scale deployment moteur-controle-fleet --replicas=5
```

Observez :

```
kubectl get pods -l app=moteur
```

✅ **Résultat** : 5 Pods au total !

Pour revenir à 3 :

```
kubectl scale deployment moteur-controle-fleet --replicas=3
```

DÉBRIEFING FINAL

Ce que vous avez appris

Concept	Rôle	Résilient ?
Pod	Unité de base (conteneur packagé)	Non
Service	Point d'accès stable avec load balancing	Oui*
Deployment	Contrôleur qui garantit X replicas	Oui (self-healing)

** Tant qu'il y a des Pods disponibles*

Questions de réflexion

Q1 : Que se passe-t-il si vous supprimez le Deployment entier ?

```
kubectl delete deployment moteur-controle-fleet
kubectl get pods
```

Q2 : Le Service existe-t-il encore ?

```
kubectl get service
```

Q3 : Quelle est la différence fondamentale entre Pod et Deployment ?

Commandes utiles pour aller plus loin

```
# Voir les logs d'un Pod
kubectl logs <nom-du-pod>

# Entrer dans un Pod (shell interactif)
kubectl exec -it <nom-du-pod> -- /bin/sh

# Voir tous les objets créés
kubectl get all

# Voir les événements récents
kubectl get events --sort-by='.lastTimestamp'
```

*Ce TP a été réalisé par **Lysa Matmar** et **Omar Samy Ahmed Hussein** de **Polytech Montpellier***

🏅 MISSION ACCOMPLIE

Félicitations ! Vous avez sauvé la station K8S-Alpha en maîtrisant :

- Les Pods (capsules)
- Les Services (standardistes)
- Les Deployments (chefs d'équipe)
- Le Self-Healing (auto-réparation)

Vos prochaines missions

- ConfigMaps & Secrets
- Ingress Controllers
- Persistent Volumes
- Helm Charts
-

AIDE RAPIDE

Erreur : command not found: minikube ou kubectl

un souci dans votre équipement ?
minikube
kubectl

Erreur : ImagePullBackOff

```
kubectl describe pod <nom-pod>
# Vérifiez que l'image existe : marcincuber/2048-game:latest
```

Erreur : Service inaccessible

```
# Vérifier que le selector correspond aux labels
kubectl describe service service-moteur
kubectl get pods --show-labels
```

Pods en CrashLoopBackOff

```
kubectl logs <nom-pod>
# Le conteneur démarre puis crash, voir les logs
```

Bonus : Profitez du jeu 2048 pendant que Kubernetes travaille pour vous ! 😄

*Répondez a ce questionnaire : **https://forms.gle/rcy6G8whS3NAyPve7***

*Ce TP a été réalisé par **Lysa Matmar** et **Omar Samy Ahmed Hussein** de **Polytech Montpellier***